MetroInsights Phase 7 - Security and Failure Analysis

Repository-based review | Revision 4405d167fc323f33be76e2c7f6614fd21cc574a8

How to read this review

This document separates verified code behavior from plausible risks that require runtime confirmation. "Verified" means the behavior is visible in the checked-in implementation; it does not mean a live exploit was executed. Reproduction steps are designed for a local disposable environment and avoid destructive actions. No application code is changed by this review.

MetroInsights is a Flask forecasting and planning application. Its prediction endpoint computes and returns analysis; the reviewed endpoint does not appear to modify persistent records. Model and metadata files are loaded with `joblib` at startup. The training pipeline consumes a large generated synthetic CSV, while request-time inference uses the fitted model, metadata, and compact station files.

Trust boundaries and assets

Main trust boundaries

• Browser input crosses into Flask via JSON or query parameters at `/api/predict`.

• Repository data files supply station names, line labels, coordinates, and operations assumptions.

• Serialized model and metadata artifacts cross from build/training time into the serving process.

• Forecast and travel-time output crosses back to the browser and is inserted into the DOM.

• The public deployment platform and dependency registry provide the runtime and packages.

Assets to protect

• Integrity of model/metadata artifacts and topology/operations files.

• Availability of the public web process under repeated or oversized requests.

• Correctness and provenance of forecasts, capacity recommendations, and estimated travel times.

• Deployment credentials and any future user or operational data. No credentials should be put in repository files.

Severity and status summary

Finding	Status	Practical concern
Station input is not checked against the supported station list	Verified behavior	Unknown names can produce plausible-looking fallback predictions and empty/incorrect route results
Untrusted values are interpolated into DOM HTML	Verified unsafe sink; exploit path depends on data/input control	Potential DOM XSS if data source becomes untrusted; search query is locally user-controlled
Joblib artifacts are deserialized at startup	Verified behavior; compromise scenario is conditional	A malicious/tampered artifact can execute code in the app process during load
No authentication or rate limiting on public calculation endpoints	Verified in reviewed routes; deployment perimeter may differ	Request flooding or repeated CPU work can affect availability
No explicit request body size limit in app configuration	Verified in app initialization	Large JSON bodies can consume memory before useful validation
Forecast scores and operational assumptions are synthetic/configurable	Verified disclosures in API/metrics	Users can mistake scenario output for measured DMRC operations
Model evaluation uses synthetic future holdout	Verified	Good synthetic holdout scores are not evidence of real-world accuracy
Dependency versions are partly open-ended and no lock file is present	Verified repository state	Rebuilds can resolve to different packages; artifact/runtime compatibility can drift
Concurrency/race failure in immutable prediction path	Not established	Must be load-tested before asserting thread-safety or race behavior

Finding 1 - Unknown station values can flow into prediction

Status: Verified behavior, not a demonstrated security exploit.

Preconditions: A caller sends an unsupported station string to `/api/predict`.

Evidence: `app/routes.py` strips punctuation and whitespace from the supplied value but does not reject it against `ALL_STATIONS`. `get_station_info()` returns an `Unknown` record for missing station keys. `predict_hourly_footfall()` then uses metadata fallbacks with index `0`; the estimator can therefore produce a numeric prediction for the wrong station identity. Route lookup may return no useful destinations. The result can look valid despite an invalid station.

Potential impact: Misleading forecasts, recommendations, and empty or misleading travel results. It also permits arbitrary station strings to reach computation repeatedly.

Actual protection: Date/time parsing catches `ValueError`; there is no equivalent supported-station validation in the reviewed route.

Safe reproduction: In a local Flask test client, request `/api/predict` with a valid date/time and a clearly nonexistent station such as `NotARealStation`. Observe whether the route returns a success response and inspect `station_info`, forecast, and travel list. This does not write data.

Recommended mitigation: Reject unknown stations with HTTP 400 before forecasting. Normalize only known aliases; return a stable error schema. Add a regression test for an unknown station and verify valid stations still work.

Finding 2 - HTML string interpolation creates a DOM injection sink

Status: Verified unsafe rendering pattern. Remote exploitability is conditional on who can control the inserted values.

Preconditions: Attacker-controlled text reaches a value interpolated into `innerHTML`.

Evidence: In `app/static/js/dashboard.js`, the empty-search message inserts the search `query` into a template string assigned to `tr.innerHTML`. The destination, line, and time fields are also interpolated into `innerHTML` for table rows. Current destination/line values are produced from repository topology/station files, so an exploit would additionally require control or compromise of those inputs; the search query itself is browser-local input.

Potential impact: Script/HTML injection in the dashboard browser context if an attacker-controlled string is rendered as markup. A user typing a crafted search term into their own page is not by itself proof of a remote attack, but the pattern becomes hazardous if the field is prefilled from a URL or data source later.

Actual protection: Jinja `tojson` is used for the station metadata injection into the page. That does not protect the later JavaScript `innerHTML` sinks. Table counts use `textContent`.

Safe reproduction: In a local browser, enter a benign markup probe such as `<b>probe</b>` into the travel search until the no-results message appears. Confirm whether it renders as literal characters or as a bold element by inspecting the DOM. Do not use executable script payloads.

Recommended mitigation: Build cells using `document.createElement()` and assign external strings with `textContent`; use fixed markup only for trusted structure. For the line color, map a known line identifier to a fixed CSS color rather than accepting arbitrary style text. Add a test that data and search values render as text.

Finding 3 - Deserializing joblib files trusts the artifact source

Status: Verified deserialization; malicious artifact execution is a conditional supply-chain threat.

Preconditions: Someone can replace or tamper with `models/footfall_forecaster.joblib` or `models/forecaster_metadata.joblib` before application startup.

Evidence: `app/forecasting.py` calls `joblib.load()` for both files during module import. Joblib/pickle-style artifacts are not safe for untrusted inputs and can execute code during deserialization.

Potential impact: Code execution with the web process's permissions, exposure of environment variables/secrets available to that process, or service compromise.

Actual protection: Artifacts are part of the repository release, which can provide source-control review if the repository and deployment credentials are protected. No artifact signature/hash verification is visible in the reviewed load path.

Safe reproduction: Do not replace the production or local model files with a crafted artifact. Verify the trust boundary by identifying which build/release process writes the artifacts and who can change that source. In a disposable copy, test startup

behavior only with known-good artifacts and intentionally missing/corrupt artifacts.

Recommended mitigation: Load only artifacts built by a trusted pipeline; restrict repository and deployment write access; pin the artifact to a release/checksum; store hashes in a manifest and verify them before loading. Fail startup clearly if the checksum or schema is wrong. Never accept model uploads from users.

Finding 4 - Public calculation endpoints can be abused for resource exhaustion

Status: No authentication or rate limit is present in the reviewed application routes; actual exposure and impact depend on hosting controls and traffic.

Preconditions: The service is reachable publicly and an attacker can send repeated or large requests.

Evidence: `/api/predict` accepts GET and POST. The app initialization does not set `MAX_CONTENT_LENGTH`; the reviewed route does not enforce a request quota or per-client rate limit. Each valid request builds a profile and runs prediction, schedule, and route work. Render or an upstream proxy may apply its own limits, but this repository does not establish them.

Potential impact: CPU/memory pressure, worker saturation, increased latency, worker restarts, or hosting quota exhaustion. A very large JSON body can consume memory during parsing before the application extracts a few fields.

Actual protection: Date and time have strict format parsing. The code path is bounded by a small network and fixed-size model; this limits per-request complexity but does not limit request rate/body size.

Safe reproduction: On a local instance only, send a small number of ordinary requests and observe request duration/RSS; separately send a body modestly larger than normal and check current behavior. Do not run flood tests against the deployed service. A production load test requires an agreed test window and rate cap.

Recommended mitigation: Set a reasonable `MAX_CONTENT_LENGTH`; validate content type, field types, and string lengths; apply upstream or app-level rate limits and concurrency limits; set worker/request timeouts; monitor 429/413/5xx rates, CPU, and memory. Keep the normal user request comfortably below limits.

Finding 5 - JSON field types are not fully validated

Status: Verified validation gap; exact exception depends on value and Flask/Werkzeug behavior.

Preconditions: Request includes a non-string `station`, `date`, `time`, or `line` value, such as JSON `null`, a number, or an array.

Evidence: The route directly calls `.strip()` on the station and passes date/time values into parsing; it assumes expected string types. The `ValueError` handler covers invalid date/time strings but not attribute/type errors raised before or outside that handler.

Potential impact: A malformed request can become an unhandled 500 instead of a clear client error. Repeated malformed requests can add noise and consume worker capacity.

Actual protection: Date and time strings are parsed against explicit formats when they are strings.

Safe reproduction: Locally POST small JSON objects with `station: null`, `station: 1`, and `time: []`; record status and whether the response is a controlled 400 or an internal error. Avoid sending huge bodies.

Recommended mitigation: Require a JSON object, validate each field's type, size, and allowed value before use, and centralize validation in a small request-schema function. Return a consistent 400 JSON error without a traceback to the client. Add tests for missing, null, wrong-type, and malformed inputs.

Finding 6 - Time/date validity is syntactic, not operationally bounded

Status: Verified behavior; impact is primarily correctness and load predictability.

Preconditions: Caller submits a syntactically valid date far outside the generated/training period or operating calendar.

Evidence: `_parse_datetime()` validates date/time syntax but accepts any parseable calendar date. The model can be asked for years beyond the synthetic data horizon, and the API returns estimates without a clear out-of-domain warning. Operating-hours logic zeros demand outside the modeled service window, but this does not address an out-of-training date during service hours.

Potential impact: Confident-looking extrapolation with unknown accuracy; calendar assumptions may also be incomplete for distant dates. The app does disclose synthetic provenance, but the selected date is not constrained to a validated horizon.

Actual protection: Date parsing rejects malformed dates; service-hours logic handles closed hours; response provenance says the forecast uses synthetic simulation, not observed counts.

Safe reproduction: Query two dates within the model's synthetic period and one valid date well beyond it; compare response status and provenance. Do not interpret response success as validation.

Recommended mitigation: Document the supported prediction horizon; flag or reject dates beyond it, or return a visible out-of-domain warning. Validate holiday/calendar coverage by year and keep a model training range in metadata.

Finding 7 - Synthetic evaluation can be mistaken for real accuracy

Status: Verified limitation, not an implementation exploit.

Preconditions: A reader interprets test metrics as validation against observed DMRC passenger counts.

Evidence: The training pipeline creates a chronological holdout from synthetic simulation data; evaluation metadata labels the dataset as a synthetic scenario test split. The serving summary sets `data_basis` to `SYNTHETIC_SIMULATION_ONLY` and `real_world_validation_available` to false. Synthetic data generation uses a fixed seed and project-authored assumptions.

Potential impact: Overstated model credibility, poor planning decisions if synthetic fit is treated as measured ridership accuracy, and misleading interview/deployment claims.

Actual protection: Dataset name, API provenance, and evaluation summary explicitly disclose synthetic basis and lack of real-world validation.

Safe reproduction: Read `models/evaluation_metrics.json`, `pipeline/02_generate_calibrated_dataset.py`, and `get_model_evaluation_summary()`. Confirm the evaluation dataset name and `real_world_validation_available` value; no retraining is needed.

Recommended mitigation: Preserve the disclosure in UI and reports; report metrics as synthetic-holdout metrics; evaluate against held-out observed counts before claiming operational accuracy. Track data source and generation parameters with every model artifact.

Finding 8 - Reproducibility and dependency drift

Status: Verified repository condition; the production impact is conditional.

Preconditions: A future build resolves a different package version or uses an artifact created under an incompatible runtime.

Evidence: `requirements.txt` has exact pins for several dependencies but ranges such as `joblib>=1.4.0` and `scikit-learn>=1.5.0`, and `gunicorn` is unpinned. No checked-in full dependency lockfile was found in the reviewed repository state.

Potential impact: Different build results, changed behavior, startup incompatibility when deserializing model artifacts, or an unreviewed dependency update.

Actual protection: Several direct dependencies are pinned; model files are deployed alongside the app revision.

Safe reproduction: In a disposable environment, compare the resolved package list from two clean installs at different dates and inspect version changes. Do not modify the live Render environment for this check.

Recommended mitigation: Use a reproducible lock or constraints file; record Python and package versions used to train artifacts; test model load and a small inference smoke test as part of release validation; update dependencies intentionally and review security advisories.

Finding 9 - Data integrity and artifact compatibility

Status: Startup failure and wrong-output risks are plausible; no signature/schema gate is visible in the serving code.

Preconditions: Station CSV schema changes, metadata feature columns do not match the model, or model and metadata from different training runs are deployed together.

Evidence: `forecasting.py` loads model, metadata, and station CSV at import, then creates a one-row feature frame using metadata `feature_cols`. Missing files or columns can fail startup/inference. The prediction code uses `values[name]`, which raises if metadata asks for an unknown feature. No explicit artifact version handshake is visible.

Potential impact: Service fails to start, returns 500s on prediction, or uses mismatched indexes/features and generates incorrect results.

Actual protection: The model feature order comes from metadata rather than a hard-coded order, reducing one class of ordering drift. This does not validate compatibility or integrity.

Safe reproduction: In a temporary copy, inspect required filenames and feature columns; use known-good files for a smoke test. A missing/corrupt artifact check can be done in a disposable environment only; do not rename or edit the working/deployed model files.

Recommended mitigation: Add an artifact manifest containing schema version, model version, feature list/hash, station-data version, and training source revision. Validate required columns and expected feature count on startup. Build a release directory and promote code/model/metadata together atomically; keep the previous release for rollback.

Finding 10 - Edge cases and availability semantics

Status: Mixed: some protections are implemented, while completeness needs testing.

Preconditions: Boundary time, unsupported station, failed routing calculation, or concurrent requests happen.

Evidence: `is_service_operating()` gates prediction outside modeled hours; `_parse_datetime()` validates formats; Flask routes include successful JSON output and error handling for invalid date/time. `/api/predict` computes travel results as part of the same response, so an unexpected graph exception may fail the whole response. There is no explicit health/readiness route in the reviewed route list.

Potential impact: Boundary errors, avoidable 500 responses, or the dashboard becoming unavailable when a nonessential travel-time section fails.

Actual protection: Closed-hour responses set demand and trains to zero; date parse failures return 400; topology graph is built once at startup rather than per request.

Safe reproduction: Locally test 04:59, 05:00, 23:29, 23:30; malformed date/time; a valid station; and an unknown station. Confirm all response fields agree with service-open state. Inspect logs for unexpected exceptions. Do not change production data.

Recommended mitigation: Add boundary tests for the service window and response invariants; decide whether route times are optional and return a partial response with a clear warning if Dijkstra fails; add readiness checks for model and graph availability. Avoid silent fallback values that appear authoritative.

Finding 11 - Privacy, authentication, and authorization scope

Status: No user authentication or role-based authorization is present in the reviewed dashboard/API. No passenger-level personal records were identified in the inspected route/data flow.

Preconditions: Product scope expands to saved user scenarios, staff-only controls, or restricted operational data.

Evidence: The reviewed API accepts scenario parameters and returns derived estimates; routes contain no login/role checks. Repository data is station-level synthetic/project data. The absence of auth is not automatically a vulnerability for a public read-only demo, but it would be insufficient for restricted write or operational actions.

Potential impact: If protected features/data are later added without access control, unauthorized viewing or changes become possible. Logging raw request bodies in a future diagnostic change could also capture unnecessary data.

Actual protection: No sensitive user data was evident in the reviewed calculation flow. This is not a replacement for a privacy assessment of future sources or deployment logs.

Safe reproduction: Inspect routes and current request/response schemas; verify no personal identifiers or state-changing operations are included in the prediction flow. No account probing or credential testing is needed.

Recommended mitigation: Keep this demo explicitly read-only. If user accounts or staff functionality are added, introduce authentication, authorization per action, CSRF protection for cookie-authenticated writes, secure session configuration, and data minimization. Define logging and retention before collecting user-linked data.

Finding 12 - Concurrency and race conditions

Status: No concrete race condition established from static review.

Preconditions: Concurrent requests execute in the same or separate Gunicorn workers.

Evidence: The reviewed prediction code reads module-level model/metadata/data and constructs request-local values. No obvious request-time writes to shared application state appear in `/api/predict`. The browser `predictionRequestId` prevents stale responses from overwriting newer UI state; it does not cancel server work. A static code read does not prove estimator thread-safety or deployment behavior under concurrency.

Potential impact: If future code mutates shared caches, model files, or global result objects without synchronization, requests could interfere. Current main risk is resource contention rather than a demonstrated data race.

Actual protection: Most inference inputs and output objects are local to a request; artifacts are loaded once and treated as read-only.

Safe reproduction: In a local, non-production environment, issue a small bounded set of concurrent identical requests and compare response consistency and error rate while observing memory/CPU. This is a controlled verification, not a stress test.

Recommended mitigation: Keep model artifacts immutable during worker lifetime; use bounded thread/process settings; synchronize or use process-safe storage for any new mutable shared cache; add a capped concurrency test before increasing workers. Do not update files in-place while serving.

Safe verification checklist

• [ ] Unknown station returns a controlled client error, not a plausible forecast.

• [ ] Wrong JSON types return a consistent 400 response.

• [ ] Large request bodies are rejected before expensive parsing/processing.

• [ ] Search and station values render as text, not executable markup.

• [ ] Missing, incompatible, or checksum-mismatched artifacts prevent readiness.

• [ ] 04:59, 05:00, 23:29, and 23:30 have consistent service/demand behavior.

• [ ] Synthetic evaluation is visibly distinguished from measured accuracy.

• [ ] Logs do not contain credentials or unnecessary personal data.

• [ ] Bounded local concurrency preserves response consistency and stays within memory budget.

• [ ] Dependency and artifact versions are recorded for each deployed release.

Interview framing

A strong security review does not call every gap an exploit. State the precondition, demonstrated behavior, affected asset, and existing protection. For example: "The API accepts an unsupported station and the forecasting layer falls back to index zero, so I would treat this as a correctness/validation bug. I would add an allow-list check and a regression test. Separately, the frontend uses `innerHTML` with dynamic strings. The current search term is locally entered, while topology values come from repository data, so remote exploitability is conditional; replacing those interpolations with `textContent` removes the unsafe sink."